



230968336

SAMEER RAJ

Week 2

Experiment 1(Solved)

A vacuum cleaner agent operates in a simple environment consisting of two rooms, labeled A and B, arranged linearly such that room A is to the left of room B.

```
In [1]: class VacuumAgent:
        def __init__(self):
            self.location = 'A'

        def act(self, percept):
            location, status = percept
            if status == 'Dirty':
                return 'suck'
            elif location == 'A':
                return 'Right'
            else:
                return 'Left'

        agent = VacuumAgent()
        print(agent.act(('A', 'Dirty')))
```

suck

Experiment 2

Extend the Simple Reflex Vacuum Cleaner Agent to operate in a 4-room grid environment, arranged as a 2×2 grid with rooms labeled (0,0), (0,1), (1,0), (1,1). Each room can independently be Clean or Dirty, with the initial dirt configuration generated randomly at the start of the simulation.

```
In [33]: import random

environment = {
    (0, 0): random.choice(["Clean", "Dirty"]),
    (0, 1): random.choice(["Clean", "Dirty"]),
    (1, 0): random.choice(["Clean", "Dirty"]),
    (1, 1): random.choice(["Clean", "Dirty"])
```

```

}

agent_location = (0, 0)

rooms_cleaned = 0
movements = 0

print("Initial Environment:")
print(environment)

while "Dirty" in environment.values():

    x, y = agent_location
    status = environment[(x, y)]

    print("Agent Location:", agent_location)
    print("Room Status:", status)

    if status == "Dirty":
        environment[(x, y)] = "Clean"
        rooms_cleaned += 1
        print("Action: Suck")

    else:
        possible_moves = []

        if x > 0:
            possible_moves.append("Up")
        if x < 1:
            possible_moves.append("Down")
        if y > 0:
            possible_moves.append("Left")
        if y < 1:
            possible_moves.append("Right")

        action = random.choice(possible_moves)
        print("Action:", action)

        if action == "Up":
            agent_location = (x - 1, y)
        elif action == "Down":
            agent_location = (x + 1, y)
        elif action == "Left":
            agent_location = (x, y - 1)
        elif action == "Right":
            agent_location = (x, y + 1)

```

```
        movements += 1

    print()

    print("All rooms are clean. Simulation stopped.")
    print("Total Rooms Cleaned:", rooms_cleaned)
    print("Total Movements:", movements)
    print("Final Environment:")
    print(environment)
```

Initial Environment:
{(0, 0): 'Clean', (0, 1): 'Clean', (1, 0): 'Dirty', (1, 1): 'Dirty'}
Agent Location: (0, 0)
Room Status: Clean
Action: Right

Agent Location: (0, 1)
Room Status: Clean
Action: Left

Agent Location: (0, 0)
Room Status: Clean
Action: Down

Agent Location: (1, 0)
Room Status: Dirty
Action: Suck

Agent Location: (1, 0)
Room Status: Clean
Action: Up

Agent Location: (0, 0)
Room Status: Clean
Action: Right

Agent Location: (0, 1)
Room Status: Clean
Action: Left

Agent Location: (0, 0)
Room Status: Clean
Action: Down

Agent Location: (1, 0)
Room Status: Clean
Action: Right

Agent Location: (1, 1)
Room Status: Dirty
Action: Suck

All rooms are clean. Simulation stopped.
Total Rooms Cleaned: 2
Total Movements: 8
Final Environment:
{(0, 0): 'Clean', (0, 1): 'Clean', (1, 0): 'Clean', (1, 1): 'Clean'}

Experiment 3

Model-Based Vacuum Cleaner Agent Problem Statement: Design and implement a

Model-Based Vacuum Cleaner Agent for the 4-room grid environment described in Experiment 2. Unlike the simple reflex agent, this agent must maintain an internal model of the environment that records the cleanliness status of rooms it has already visited.

```
In [34]: environment = {
    (0, 0): random.choice(["Clean", "Dirty"]),
    (0, 1): random.choice(["Clean", "Dirty"]),
    (1, 0): random.choice(["Clean", "Dirty"]),
    (1, 1): random.choice(["Clean", "Dirty"])
}

model = {
    (0, 0): "Unknown",
    (0, 1): "Unknown",
    (1, 0): "Unknown",
    (1, 1): "Unknown"
}

agent_location = (0, 0)

rooms_cleaned = 0
movements = 0

print("Initial Environment:")
print(environment)

while "Unknown" in model.values() or "Dirty" in model.values():

    x, y = agent_location
    percept_status = environment[(x, y)]

    print("Agent Location:", agent_location)
    print("Perceived Status:", percept_status)

    model[(x, y)] = percept_status

    if percept_status == "Dirty":
        environment[(x, y)] = "Clean"
        model[(x, y)] = "Clean"
        rooms_cleaned += 1
        print("Action: Suck")
```

```

else:
    possible_moves = []

    if x > 0 and model[(x - 1, y)] != "Clean":
        possible_moves.append(("Up", (x - 1, y)))
    if x < 1 and model[(x + 1, y)] != "Clean":
        possible_moves.append(("Down", (x + 1, y)))
    if y > 0 and model[(x, y - 1)] != "Clean":
        possible_moves.append(("Left", (x, y - 1)))
    if y < 1 and model[(x, y + 1)] != "Clean":
        possible_moves.append(("Right", (x, y + 1)))

    if not possible_moves:
        if x > 0:
            possible_moves.append(("Up", (x - 1, y)))
        if x < 1:
            possible_moves.append(("Down", (x + 1, y)))
        if y > 0:
            possible_moves.append(("Left", (x, y - 1)))
        if y < 1:
            possible_moves.append(("Right", (x, y + 1)))

    action, new_location = random.choice(possible_moves)
    agent_location = new_location
    movements += 1
    print("Action:", action)

    print("Internal Model:", model)
    print()

print("All rooms are clean and known.")
print("Total Rooms Cleaned:", rooms_cleaned)
print("Total Movements:", movements)
print("Final Environment:")
print(environment)

```

Initial Environment:
 {(0, 0): 'Clean', (0, 1): 'Clean', (1, 0): 'Clean', (1, 1): 'Dirty'}
 Agent Location: (0, 0)
 Perceived Status: Clean
 Action: Down
 Internal Model: {(0, 0): 'Clean', (0, 1): 'Unknown', (1, 0): 'Unknown', (1, 1): 'Unknown'}

Agent Location: (1, 0)
 Perceived Status: Clean
 Action: Right
 Internal Model: {(0, 0): 'Clean', (0, 1): 'Unknown', (1, 0): 'Clean', (1, 1): 'Unknown'}

Agent Location: (1, 1)
 Perceived Status: Dirty
 Action: Suck
 Internal Model: {(0, 0): 'Clean', (0, 1): 'Unknown', (1, 0): 'Clean', (1, 1): 'Clean'}

Agent Location: (1, 1)
 Perceived Status: Clean
 Action: Up
 Internal Model: {(0, 0): 'Clean', (0, 1): 'Unknown', (1, 0): 'Clean', (1, 1): 'Clean'}

Agent Location: (0, 1)
 Perceived Status: Clean
 Action: Left
 Internal Model: {(0, 0): 'Clean', (0, 1): 'Clean', (1, 0): 'Clean', (1, 1): 'Clean'}

All rooms are clean and known.
 Total Rooms Cleaned: 1
 Total Movements: 4
 Final Environment:
 {(0, 0): 'Clean', (0, 1): 'Clean', (1, 0): 'Clean', (1, 1): 'Clean'}

End Questions

Question: Why does the simple reflex agent perform redundant actions in the grid environment?

Answer: The simple reflex agent performs redundant actions because it does not have memory. It makes decisions based only on the current percept (current location and room status). As a result, the agent cannot remember which rooms were already cleaned, so it may revisit and move through clean rooms multiple times.

Question: How does the internal model improve agent performance?

Answer: The internal model improves agent performance by storing the cleanliness status of visited rooms. Using this information, the agent avoids revisiting rooms that are already known to be clean, prefers moving toward unvisited or dirty rooms. This reduces unnecessary movements and allows the agent to clean all rooms more efficiently.

Question: Compare the simple reflex and model-based agents

Rationality

The simple reflex agent is less rational because it makes decisions only based on the current percept and does not consider past experiences. It cannot distinguish between new and already cleaned rooms.

The model-based agent is more rational because it maintains an internal model of the environment and uses past percepts to make better decisions.

Efficiency

The simple reflex agent is less efficient since it frequently revisits clean rooms and performs redundant movements.

The model-based agent is more efficient because it avoids unnecessary revisits and reaches dirty or unvisited rooms more quickly.

Scalability

The simple reflex agent does not scale well in larger or more complex environments because the number of redundant actions increases significantly.

The model-based agent scales better since its internal model helps it manage and navigate larger environments more effectively.

Question: Would the model-based agent still work correctly in a partially observable environment? Justify.

Answer: Yes, the model-based agent can still work in a partially observable environment, but with limitations. Since the agent updates its model only using its own percepts, missing or incorrect percepts may lead to an inaccurate internal model. However, the agent can still make better decisions than a simple reflex agent by using whatever information is available.